The Essential of ML: Chapter 5

Combining models, cut down to what Tutorial 2 tests: the variance of an average, votes, Gini splits, bootstrap and m_{try} probabilities, importance credits, residuals with shrinkage, Newton leaves, the XGBoost gain

Fundamentals of Machine Learning

2026-10-30

How to read this document

Chapter 5 with everything removed that Tutorial 2 does not touch. Seven sections in concept order, every term defined before use, every formula followed by a worked number. Each section ends with a *Tutorial drill*: the Tutorial 2 questions that live there, answered in the rubric's shape (claim, mechanism, number or derivation, consequence), plus a variant with new numbers. Left out on purpose: multi-class boosting, the library zoo (LightGBM, CatBoost, AdaBoost), the completing-the-square proof for XGBoost, Lab 3. Numbers to carry: $e^{-1} = 0.368$, $1 - e^{-1} = 0.632$, Gini of a 50/50 node $= 0.5$, $h = \hat{p}(1-\hat{p}) \leq 0.25$.

1 Why Combine Models: Averaging, Voting and the Variance Formula

Ensemble, averaging, hard and soft voting

An **ensemble** fits M models $\hat{f}_1, \ldots, \hat{f}_M$ and combines their predictions by one rule.

- **Averaging** (regression): $\hat{f}_{\text{ens}}(x) = \frac{1}{M}\sum_{m=1}^{M} \hat{f}_m(x)$.
- **Hard voting** (classification): each model outputs a class $\hat{y}_m(x)$ (its probability thresholded at 0.5), one vote each; the ensemble outputs the most frequent class.
- **Soft voting**: average the posterior estimates and take the largest average, $\hat{p}_{\text{ens}}(y = c \mid x) = \frac{1}{M}\sum_m \hat{p}_m(y = c \mid x)$, $\hat{y} = \arg\max_c \hat{p}_{\text{ens}}$.

Hard voting counts heads; soft voting pools **margins**, the distance of each probability from the threshold: 0.49 and 0.05 are the same hard vote and very different soft votes.

Soft voting uses strictly more information, so it is the better rule *when the probabilities deserve to be believed*: reasonably **calibrated** (a model that says 0.95 is right about 95% of the time) and under the same class prior. An **overconfident** model, outputting 0.95 or 0.05 whenever it leans one way, moves a three-model average by about ± 0.15 alone and dominates every soft vote; hard voting caps it at one vote, robust at the price of discarding margins. Slide 11: $0.49, 0.48, 0.95$ is hard **0** (two to one) and soft **1** (0.64); which is right depends on whether C's 0.95 is calibrated.

Why averaging helps: the variance of an average of correlated errors

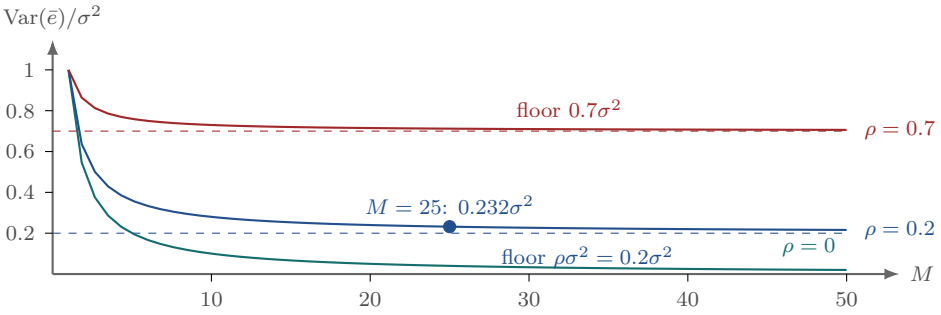
Fix an input x and let $e_m = \hat{f}_m(x) - f(x)$ be the error of model m, a random variable over training samples. Suppose $\text{Var}(e_m) = \sigma^2$ for every m and $\text{Corr}(e_m, e_{m'}) = \rho$ for every pair $m \neq m'$. The

averaged error $\bar{e} = \frac{1}{M}\sum_m e_m$ has

$$\mathrm{Var}(\bar{e}) = \frac{1}{M^2}\Big[\underbrace{M\sigma^2}_{M \text{ variances}} + \underbrace{M(M-1)\rho\sigma^2}_{M(M-1) \text{ covariances}}\Big] = \frac{\sigma^2}{M} + \frac{(M-1)\rho\sigma^2}{M} = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big).$$

Derivation. $\mathrm{Var}(\sum_m e_m) = \sum_m \mathrm{Var}(e_m) + \sum_{m\neq m'} \mathrm{Cov}(e_m, e_{m'})$; there are M diagonal terms worth σ^2 and $M(M-1)$ ordered pairs worth $\rho\sigma^2$ each; divide by M^2 and regroup: $\frac{\sigma^2}{M} + \rho\sigma^2 - \frac{\rho\sigma^2}{M} = \sigma^2(\rho + \frac{1-\rho}{M})$.

Three cases. $\rho = 0$: σ^2/M, averaging divides the variance by M. $\rho = 1$: σ^2, averaging changes nothing. $M \to \infty$: the variance tends to $\rho\sigma^2$, a **floor** that no number of models removes.



Key insight. The $M(M-1)$ cross terms carry ρ and do not shrink after division by M^2: the part of the error all models *share* is immune to averaging, so errors must differ. Averaging is a variance tool, not a bias tool (every model missing the same feature means $\rho \approx 1$). Every method below lowers ρ without raising σ^2 too much.

Tutorial drill

Q1 (Tutorial 2, Q1(a)). Three classifiers, threshold 0.5, four flows: $(p_A, p_B, p_C, y) = (0.49, 0.49, 0.95, 1)$, $(0.80, 0.55, 0.05, 1)$, $(0.40, 0.40, 0.90, 0)$, $(0.10, 0.70, 0.70, 1)$. Hard and soft votes; where they disagree, what does one rule use that the other discards?

Answer. Threshold each p to a vote and count; average the three p and threshold the average. Flow 1: votes $0, 0, 1$, hard **0**; $\bar{p} = 1.93/3 = 0.643$, soft **1**. Flow 2: $1, 1, 0$, hard **1**; $\bar{p} = 1.40/3 = 0.467$, soft **0**. Flow 3: $0, 0, 1$, hard **0**; $\bar{p} = 1.70/3 = 0.567$, soft **1**. Flow 4: $0, 1, 1$, hard **1**; $\bar{p} = 0.500$, a tie, soft **1** under "$\geq 0.5 \Rightarrow$ attack" (state it). Hard is right on 2, 3, 4; soft on 1, 4. Every disagreement is about the *margin*: soft uses the distance of each p from 0.5 (two coin flips against C's 0.95), hard treats 0.49 and 0.05 identically. Soft wins when the confident model deserves its confidence (flow 1), loses when it does not (flows 2, 3).

Q2 (Tutorial 2, Q1(b)). C is often too confident. Effect on each rule? Is hard voting therefore always better? Counterexample.

Answer. Soft voting: C's 0.95/0.05 moves the average by about ± 0.15, A and B together by at most ± 0.07, so C *dominates*: every disagreement above was decided by C, and when C is wrong (flows 2, 3) it drags the ensemble with it; the calibration assumption is violated. Hard voting caps C at one vote, so A and B can outvote it: robust to one miscalibrated member. Not always better, because it is robust *by discarding information*. Counterexample: two models at 0.51 (wrong), one calibrated at 0.99 (right): hard is wrong 2–1; soft gives $(0.51 + 0.51 + 0.01)/3 = 0.343$, right. Remedy: calibrate C or learn weights on validation data (stacking, §7). "Hard won 3–2 here" is no evidence: four flows is not a validation set.

Q3 (variant, new numbers). Ten regressors each have error variance $\sigma^2 = 4$ at some input. Variance of the average for $\rho = 0$, 0.3, 0.9; the limit as $M \to \infty$ at $\rho = 0.3$; the smallest M that brings it to 1.6 or below.

Answer. Var $= 4(\rho + (1-\rho)/10)$. $\rho = 0$: 0.4. $\rho = 0.3$: $4(0.3 + 0.07) = 1.48$. $\rho = 0.9$: $4(0.9 + 0.01) = 3.64$, barely better than one model. Limit $4\rho = 1.2$. $4(0.3 + 0.7/M) \leq 1.6 \iff 0.7/M \leq 0.1 \iff M \geq 7$. At $\rho = 0.9$ ten models buy almost nothing; the only lever left is lowering ρ, which is what bagging and forests do.

2 Decision Trees as Base Learners

Decision tree, leaf constant, depth

A tree with J leaves partitions the feature space into disjoint axis-aligned boxes $S_1, \dots, S_J$ and predicts one constant per box: $T(x) = \sum_{j=1}^{J} \theta_j \, \mathbf{1}[x \in S_j]$. Each box is reached by a path of binary tests $x_j \le t$; the **depth** of a leaf is the number of tests on its path; **maximum depth** and **minimum leaf size** are the size controls. A depth-one tree (one split, two leaves) is a **stump**. For fixed regions the objective separates by leaf, $\theta_j^* = \arg\min_\theta \sum_{i \in S_j} \ell(y^{(i)}, \theta)$: the **mean** under squared error ($\frac{d}{d\theta} \sum (y-\theta)^2 = -2 \sum (y-\theta) = 0$), the **median** under absolute error, the **majority class** under zero–one loss.

Choosing the regions jointly is infeasible, so trees are grown **greedily**: at each node take the single split that reduces impurity most, then recurse into the children until a stopping rule (depth, leaf size, no useful split). Greedy means best immediate gain, no look-ahead, no revision of an early split.

Impurity: Gini and entropy; the impurity decrease of a split

For a node S with class proportions $\hat{p}_c(S)$:

$$\text{Gini} \quad I(S) = 1 - \sum_c \hat{p}_c(S)^2, \qquad \text{entropy} \quad H(S) = -\sum_c \hat{p}_c(S) \log_2 \hat{p}_c(S).$$

Gini is the probability that two observations drawn from the node have different classes. Both are 0 for a pure node and maximal for a balanced one: two classes at 50/50 give $I = 1 - \frac{1}{4} - \frac{1}{4} = 0.5$ and $H = 1$ bit. For regression the within-node squared error $\sum_{i \in S} (y^{(i)} - \bar{y}_S)^2$ plays the same role. A split of S into S_L, S_R has **impurity decrease**

$$I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R),$$

parent minus the *size-weighted* average of the children: a tiny pure child counts little. The greedy rule picks (j^*, t^*) maximising this decrease over all features and thresholds.

Lecture 5, slide 26: choosing a split by hand

Eight observations sorted by $x = 1, \dots, 8$ with $y = 0, 0, 0, 1, 0, 1, 1, 1$. Parent: four of each, $I(S) = 0.50$.

- $x \le 3.5$: left $\{0, 0, 0\}$, $I = 0$; right $\{1, 0, 1, 1, 1\}$, $I = 1 - 0.8^2 - 0.2^2 = 0.32$. Weighted $\frac{3}{8}(0) + \frac{5}{8}(0.32) = 0.20$. Decrease $0.50 - 0.20 = \mathbf{0.30}$.
- $x \le 4.5$: left $\{0, 0, 0, 1\}$ and right $\{0, 1, 1, 1\}$, each $I = 1 - 0.75^2 - 0.25^2 = 0.375$. Weighted 0.375. Decrease $\mathbf{0.125}$.

Greedy takes $x \le 3.5$: a pure leaf beats a balanced split. Entropy agrees: $H(0.8, 0.2) = 0.722$, weighted $\frac{5}{8}(0.722) = 0.451$, decrease 0.549 bits, against $H(0.75, 0.25) = 0.811$ and a decrease of 0.189 bits.

Key insight. Trees are the base learner everywhere here because they are flexible (a test on x_1 then on x_2 *is* an interaction), scale-free and cheap to fit hundreds of times. Their weakness is **variance**: one changed observation can change an early split, and everything below it is rebuilt. Deep trees: low bias, high variance; averaging keeps the first and removes the second. Hence bagging grows trees deep, boosting (which attacks bias) grows them shallow.

Tutorial drill

Q1 (Tutorial 2, Q2(b), the tree half). Why does a tree whose root cannot use the strongest feature x_1 predict less accurately on its own?
Answer. Claim: weaker, because trees are greedy and hierarchical. Its root must take the best of weaker candidates, so the first split removes less impurity; every later split lives inside the two regions that split

created and can only refine them, never undo them. The tree fits less well and its error variance σ^2 rises a little. x_1 is not lost: it is offered at a fraction m_{try}/d of the deeper nodes and used there. Finished in §3: a slightly larger σ^2 buys a much smaller ρ.

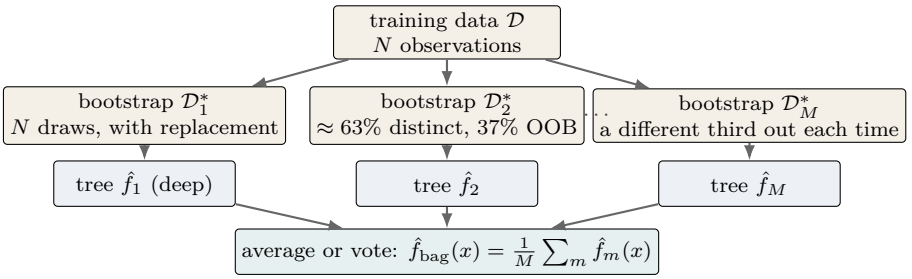
Q2 (variant, new numbers). A node holds 40 examples, $20^+/20^-$. Split A: left $(15^+, 5^-)$, right $(5^+, 15^-)$. Split B: left $(10^+, 0^-)$, right $(10^+, 20^-)$. Gini of node and children, weighted child impurity, decrease, greedy choice; what each chosen child predicts and with what posterior.

Answer. Node 0.5. A: both children $(0.75, 0.25)$, $I = 0.375$; weighted 0.375; decrease 0.125. B: left pure, $I = 0$; right $(\frac{1}{3}, \frac{2}{3})$, $I = 1 - \frac{1}{9} - \frac{4}{9} = 0.444$; weighted $\frac{10}{40}(0) + \frac{30}{40}(0.444) = 0.333$; decrease 0.167. Greedy chooses B. Left predicts positive with $\hat{p} = 1.0$ (from 10 examples, overconfident); right predicts negative with $\hat{p}(\text{pos}) = 1/3$.

3 Bagging, Out-of-Bag Evaluation and Random Forests

Bootstrap sample, bagging

Splitting $\mathcal{D}$ into M disjoint parts would give each model only N/M observations. Instead each model gets a **bootstrap sample** $\mathcal{D}^*$: N draws from $\mathcal{D}$ *with replacement*. count(i) is how often observation i appears; observations with count$(i) = 0$ are **out of bag** (OOB) for that sample. **Bagging** (bootstrap aggregating) draws M bootstrap samples, fits the *same* learner to each, and averages (regression) or averages probabilities / votes (classification): $\hat{f}_{\text{bag}}(x) = \frac{1}{M}\sum_m \hat{f}_m(x)$. Example: from $\{1, \ldots, 8\}$ the draws $\{2, 5, 2, 8, 1, 5, 7, 2\}$ give counts $1, 3, 0, 0, 2, 0, 1, 1$: observation 2 is seen three times; $3, 4, 6$ are out of bag.



random forest: at every node of every tree, offer only m_{try} random features

Out-of-bag fraction: $(1 - 1/N)^N \to e^{-1} \approx 0.368$

One draw picks observation i with probability $1/N$, so misses it with probability $1 - 1/N$. The N draws are independent, so

$$\mathbb{P}(\text{count}(i) = 0) = \left(1 - \frac{1}{N}\right)^N \xrightarrow[N\to\infty]{} e^{-1} \approx 0.368,$$

by $\lim(1 + x/N)^N = e^x$ at $x = -1$. Already $(7/8)^8 = 0.344$ for $N = 8$ and 0.366 for $N = 100$. Hence each bootstrap sample holds about 63.2% of the *distinct* observations and leaves 36.8% out, a *different* 36.8% for each model. The "in" part makes the models differ; the "out" part is free validation data.

Out-of-bag prediction and OOB error

For training observation i, let $\mathcal{M}_i = \{m : i \notin \mathcal{D}_m^*\}$ be the models that never saw i; $|\mathcal{M}_i| \approx 0.368M$. The **OOB prediction** uses only them, $\hat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\mathcal{M}_i|}\sum_{m\in\mathcal{M}_i} \hat{f}_m(x^{(i)})$; the **OOB error** is the loss of these predictions over all N training observations. A genuine unseen-data estimate without holding anything out (slightly pessimistic: a third of the trees vote per point), good for comparing *ensemble-wide* choices (tree size, m_{try}, feature set, M). Limits: repeated OOB-based selection overfits the OOB estimate like repeated validation does, so the final number needs an untouched test set; and OOB says nothing about deployment shift.

Tune the ensemble, not the trees

Tree size ξ (maximum depth, minimum leaf size) is one *ensemble* hyperparameter, $\mathcal{D}_m^* \to \widehat{T}_m(x;\xi)$ with the same ξ for every m: for each candidate ξ fit a complete bagged ensemble and compare OOB errors. Bagging usually wants deep trees; on noisy or small data OOB may favour larger leaves. Tuning each tree separately optimises the wrong target (one tree's accuracy, not the average's).

Random forest and m_{try}

Bagged trees can stay too alike: a very strong x_1 sits at the root of almost every bootstrap tree, the trees have similar shapes, their errors stay correlated (ρ large), more trees stop helping early. A **random forest** adds one step: at *each node*, draw a fresh random subset of m_{try} of the d features and split on the best of those only. $m_{\text{try}} = d$ is bagging. Start near $m_{\text{try}} \approx \sqrt{d}$ for classification ($d/3$ for regression): $d = 16$ gives four candidates per node.

Probability that a feature is offered

Choosing m_{try} of d features uniformly without replacement, a given feature is in the subset with probability m_{try}/d (count: $\binom{d-1}{m_{\text{try}}-1}/\binom{d}{m_{\text{try}}} = m_{\text{try}}/d$). It is **unavailable at one split** with probability $1 - m_{\text{try}}/d$, and, the roots being drawn independently, **unavailable at all M roots** with probability $(1 - m_{\text{try}}/d)^M$. Expected number of roots that offer it: $M\, m_{\text{try}}/d$.

Strength versus correlation, read through the variance formula

Large m_{try} (towards bagging)	Small m_{try}
strongest features reused at most nodes; each tree strong, small σ^2	alternative features get used; each tree weaker, larger σ^2
trees similar, errors correlated, large ρ	different early splits, small ρ; too few candidates can give a poor split

Forest variance $= \sigma^2(\rho + (1-\rho)/M)$. Feature randomisation raises σ^2 a little and lowers ρ a lot. Numbers: σ^2 up 10%, ρ from 0.6 to 0.3, $M = 100$: $\sigma^2(0.6 + 0.004) = 0.604\sigma^2$ becomes $1.1\sigma^2(0.3 + 0.007) = 0.338\sigma^2$. Weaker trees, stronger forest: the target is the average's error, not any tree's.

Controls. M: OOB error falls then plateaus; more trees stabilise, they do not deepen the model, so add trees until the plateau. m_{try}, leaf size, depth: tune at *forest level* by OOB or validation error, each candidate requiring a complete forest.

Tutorial drill

Q1 (Tutorial 2, Q2(a)). $d = 25$, $m_{\text{try}} = 5$, $M = 10$. Probability that x_1 is available at one split; probability that it is unavailable at the roots of all ten trees.
Answer. Available: $m_{\text{try}}/d = 5/25 = 0.2$ (by symmetry, or $\binom{24}{4}/\binom{25}{5} = 5/25$). Unavailable at one root: 0.8; roots are drawn independently, so at all ten: $0.8^{10} \approx 0.107$. Expected roots offering x_1: $10 \times 0.2 = 2$; bagging would offer it at all ten and use it at most.

Q2 (Tutorial 2, Q2(b)). Why can hiding x_1 from many roots lower each tree's accuracy yet improve the forest?
Answer. Each tree is weaker (§2 drill Q1): σ^2 up a little. The forest's error is the error of the *average*, $\sigma^2(\rho + (1-\rho)/M)$: trees that all start with x_1 make correlated mistakes and $\rho\sigma^2$ cannot be averaged away; trees that start differently partition the space differently and ρ falls. Numbers as in the trade-off box: $0.604\sigma^2 \to 0.338\sigma^2$. "Weaker trees, stronger forest" says the correlation term dominates once M is large.

Q3 (Tutorial 2, Q2(c)). How to choose m_{try} and tree size; why not tune each tree separately?
Answer. Start at $m_{\text{try}} \approx \sqrt{d} = 5$ and one shared leaf size; for each candidate pair on a small grid fit a *complete* forest and compute its OOB error (free: each observation is predicted by the $\approx 37\%$ of trees that never saw it) or a validation error; keep the best; raise M until the OOB error plateaus; keep a test set untouched, since repeated OOB selection is optimistic. Not per tree: (i) the objective is the forest's error,

and a tree tuned alone picks $m_{\text{try}} = d$ and a deep fit, recreating the correlated bagged forest; (ii) ξ is an ensemble hyperparameter, one ξ for all $\mathcal{D}_m^* \to \widehat{T}_m(x;\xi)$; (iii) OOB estimates the error of the procedure "M exchangeable trees with ξ", and M separate selections contaminate it.

Q4 (variant, new numbers). $N = 1{,}000$, $M = 500$, $d = 400$ of which 5 carry signal. Expected distinct observations per bootstrap; expected OOB trees per observation; probability an observation has no OOB prediction, for $M = 500$ and $M = 10$; probability a split sees at least one informative feature for $m_{\text{try}} = 20$ and 100.

Answer. $0.632 \times 1{,}000 \approx 632$ distinct. OOB trees: $0.368 \times 500 \approx 184$. No OOB prediction (in-bag for every tree): $0.632^{500} \approx 0$; for $M = 10$, $0.632^{10} = 0.010$. Informative feature seen: $1 - (1 - 5/400)^{m_{\text{try}}}$ approximately; $m_{\text{try}} = 20$: $1 - 0.9875^{20} = 0.22$; $m_{\text{try}} = 100$: $1 - 0.9875^{100} = 0.72$. At $\sqrt{d} = 20$ four splits in five see pure noise; the loss of strength outweighs the decorrelation, so expect 100 to win: $\sqrt{d}$ assumes signal is not this rare and is a starting point for OOB tuning, not a rule.

4 Feature Importance

Split credit and mean decrease in impurity (MDI)

For a node t holding N_t of the N training observations, split into children of sizes N_L, N_R, the **credit** of the split is its impurity decrease weighted by the fraction of data it acts on,

$$\Delta_t = \frac{N_t}{N}\Big[I(S_t) - \frac{N_L}{N_t} I(S_L) - \frac{N_R}{N_t} I(S_R)\Big],$$

and goes to the feature used at t. The **MDI** of feature j adds all credits of splits on x_j, averages over trees, and is normalised to sum to one over features: $\text{MDI}_j = \frac{1}{M}\sum_m \sum_{t \in T_m:\ \text{split}(t)=j} \Delta_t$. The bracket is the decrease of §2; the weight N_t/N says a decrease on all the data is worth more than the same decrease on 5% of it. The root has weight 1; a deeper node has a smaller weight but can earn more through a larger decrease.

Lecture 5, slides 35–36: one credit, then forest-level MDI

Root with 10 flows, 5 attacks, Gini 0.50. "Port = 22?" sends 4 left (0 attacks, Gini 0) and 6 right (5 attacks, 1 benign, Gini $1 - (5/6)^2 - (1/6)^2 = 0.278$). Weighted children $\frac{4}{10}(0) + \frac{6}{10}(0.278) = 0.167$; $N_t/N = 1$; $\Delta_t = 0.50 - 0.167 = 0.333$ to destination port.

Two trees: credits port $0.18, 0.12$; duration 0.06 (tree 1 only); packets 0.04 (tree 2 only). Means over two trees $0.15, 0.03, 0.02$, sum 0.20; normalised MDI $0.75, 0.15, 0.10$: port produced 75% of the impurity the forest removed.

MDI is free (computed during training) and biased: a feature with many split points (continuous, high-cardinality) has more chances to find *some* decrease, by signal or by noise, and is over-credited; and it measures the *training* fit, which deep trees overfit.

Permutation importance: the error increase after shuffling one feature

For the fitted forest and a feature j: (1) compute the original OOB or validation error; (2) shuffle the values of x_j among the OOB observations, keeping all other columns and the labels fixed; (3) predict again. The importance is the increase,

$$I_j = \widehat{\text{Err}}_{\text{shuffled}(j)} - \widehat{\text{Err}}_{\text{original}}.$$

No tree is retrained. Shuffling destroys the *information* in x_j while keeping its marginal distribution, so the trees see plausible values; it measures how much the fitted model *relies* on x_j in prediction, on data the trees did not see. Slide 39: 100 OOB flows, 10 errors; shuffling port gives 18 (+0.08), duration 13 (+0.03), packets 11 (+0.01).

Two readings importance does not support

- **Correlated features mask each other.** If x_a and x_b carry the same information, shuffling x_a alone hurts little: the forest still has it through x_b. Each looks less important than the information it carries. Permute them jointly, or drop a column and refit. MDI has the same problem.
- **Importance is not causation.** A high score says the fitted model's predictions change when the feature changes: a fact about the model, not the world. The feature may be a proxy (port for the service attacked), a leaked consequence of the label, or a collection artefact. Causation is about intervention.

Tutorial drill

Q1 (Tutorial 2, Q3(a)). A root (all N) reduces Gini from 0.50 to 0.45; a deeper node with $0.30N$ observations reduces its impurity from 0.48 to 0.08. Credits; why can the deeper split earn more?
Answer. "From a to b" gives the bracket directly (b is the size-weighted child impurity; state this). Root: $\Delta = \frac{N}{N}(0.50 - 0.45) = 0.05$. Deeper: $\Delta = 0.30(0.48 - 0.08) = 0.30 \times 0.40 = 0.12$, more than twice the root. The credit is a product: a small decrease applied to everyone removes $0.05N$ units of impurity; a large decrease on 30% of the data removes $0.40 \times 0.30N = 0.12N$. The weight N_t/N makes MDI count impurity removed per training observation, not per split; early splits are coarse, deep splits can nearly purify their node.

Q2 (Tutorial 2, Q3(b)). Validation error 0.12; after permuting x_a it is 0.20, after x_b it is 0.14. Permutation importances and what they mean.
Answer. $I_a = 0.20 - 0.12 = 0.08$, $I_b = 0.14 - 0.12 = 0.02$. Destroying x_a costs eight extra errors per hundred, two thirds of the baseline rate; x_b two per hundred: the fitted forest relies on x_a about four times as heavily. These are statements about *this forest's reliance in prediction*, measured on unseen data; not about how much information the features "contain". 0.02 may be noise at this validation size: repeat the shuffle and average.

Q3 (Tutorial 2, Q3(c)). x_a and x_b strongly correlated. Why does shuffling one understate it; why is importance not causation?
Answer. After shuffling x_a the shared information is still available through x_b (trees that split on x_b are untouched), so the error rises only by the part *unique* to x_a; the 0.08 vs 0.02 asymmetry says which one the greedy splitter happened to pick more often, an accident of m_{try} draws. Remedy: permute jointly, or drop and refit. Causation: importance is reliance under observation; the feature may be a proxy, a leaked consequence of the label, or a collection artefact; intervening on it would not change the outcome.

Q4 (variant, new numbers). Root reduces Gini $0.50 \to 0.42$; a node with $0.25N$ reduces $0.45 \to 0.05$. Baseline error 0.10; shuffling x_a: 0.18; x_b: 0.11; both jointly: 0.25.
Answer. Root $\Delta = 1(0.08) = 0.08$; deeper $\Delta = 0.25 \times 0.40 = 0.10$, again the deeper split wins. $I_a = 0.08$, $I_b = 0.01$, joint $0.15 > 0.08 + 0.01 = 0.09$: the joint increase exceeds the sum of the singles by 0.06, the signature of shared information that each single shuffle left available through the other feature. Report the pair.

5 Gradient Boosting for Regression

Additive boosted model, learning rate

Bagging fits models in parallel and averages whole predictions. **Boosting** asks what the next model should learn from the current mistakes: the observations stay fixed, each new tree is fitted to what the ensemble so far gets wrong, and the corrections are *summed*. After m rounds

$$F_m(x) = F_{m-1}(x) + \eta\, T_m(x), \qquad F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} T_m(x),$$

with T_m the tree of round m and $\eta \in (0, 1]$ the **learning rate** or **shrinkage**: $\eta = 1$ applies the full correction, $\eta = 0.1$ one tenth of it. A sum of corrections, not an average of predictions.

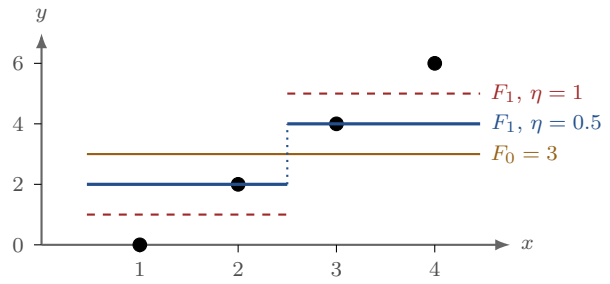
Squared loss: the next tree fits the residuals, which are the negative gradient

With $L = \frac{1}{2}\sum_i (y^{(i)} - F_m(x^{(i)}))^2$ and $F_m = F_{m-1} + \eta T_m$,

$$L = \frac{1}{2}\sum_{i=1}^{N} \Big(\underbrace{y^{(i)} - F_{m-1}(x^{(i)})}_{r_m^{(i)} \text{ fixed at round } m} - \eta T_m(x^{(i)}) \Big)^2,$$

so minimising L over T_m is least-squares fitting of ηT_m to the current **residuals**. In general the tree is fitted to the **pseudo-residual** $r_m^{(i)} = -\partial \ell(y^{(i)}, F)/\partial F$ at $F = F_{m-1}(x^{(i)})$, the direction in which moving the prediction lowers the loss fastest; for $\ell = \frac{1}{2}(y - F)^2$ this is $y - F$, so "fit the residuals" and "fit the negative gradient" coincide. (For absolute loss it would be $\text{sign}(y - F)$.)

Algorithm. (1) $F_0 = \arg\min_w \sum_i \ell(y^{(i)}, w) = \bar{y}$ for squared loss (leaf-mean rule of §2). (2) For $m = 1, \ldots, M$: $r_m^{(i)} = y^{(i)} - F_{m-1}(x^{(i)})$; fit a regression tree to the r_m, regions $S_{q,m}$, each leaf outputting the **mean residual** $w_{q,m} = \frac{1}{|S_{q,m}|}\sum_{i \in S_{q,m}} r_m^{(i)}$; update $F_m = F_{m-1} + \eta T_m$. The tree is fitted to the full residuals; η is applied in the update.



Tutorial 2, Q4. The stump T_1 outputs -2 left of 2.5 and $+2$ right. Full step ($\eta = 1$): $F_1 = (1, 1, 5, 5)$, residuals $(-1, 1, -1, 1)$ look like noise. Half step: $F_1 = (2, 2, 4, 4)$, residuals $(-2, 0, 0, 2)$ still carry the slope inside each leaf for T_2 to find.

Why many shallow trees, and why $\eta < 1$ helps validation

One deep tree fits the whole rule at once: early splits lock in rigid boundaries and are never revised. Boosted shallow trees: tree 1 isolates the loudest pattern, later trees target leftovers *anywhere* in feature space, η makes convergence gradual. Each shallow tree has high bias, low variance; the sum lowers bias while η and early stopping hold variance. Training loss falls forever; validation loss turns up when new trees fit noise: **early stopping** takes the round at the validation minimum. Three coupled controls: rounds M, rate η ($\eta \propto 1/M$: halve η, double the rounds), depth (depth 1: additive effects; depth 2: pairwise interactions; 2–6 usual). Search (η, depth), choose M for each by early stopping, compare at the selected round; never tune one alone.

Tutorial drill

Q1 (Tutorial 2, Q4(a)–(b)). $x = 1, 2, 3, 4$; $y = 0, 2, 4, 6$; squared loss; $F_0 = \bar{y}$; stump splits at $x = 2.5$; $\eta = 0.5$. Residuals, leaf means, T_1; F_1, new residuals, squared-error sums before and after.
Answer. $F_0 = (0 + 2 + 4 + 6)/4 = 3$. $r_1 = y - 3 = (-3, -1, 1, 3)$. Left leaf $\{1, 2\}$: $w_L = (-3 - 1)/2 = -2$; right $\{3, 4\}$: $w_R = (1 + 3)/2 = +2$; $T_1(x) = -2$ for $x \le 2.5$, $+2$ otherwise. $F_1 = 3 + 0.5\,T_1 = (2, 2, 4, 4)$. $r_2 = y - F_1 = (-2, 0, 0, 2)$. Sum of squares $9 + 1 + 1 + 9 = 20$ before, $4 + 0 + 0 + 4 = 8$ after (loss $\frac{1}{2}\sum r^2$: $10 \to 4$). The full step $\eta = 1$ would reach 4 (figure): the unshrunk leaf mean is the least-squares optimum, so it must remove more training error this round.

Q2 (Tutorial 2, Q4(c); also Q7(d)). Why can leaving error for later trees improve validation; why does a smaller η select more rounds, and does that make it always better?
Answer. (i) Each tree commits less: with $\eta = 1$ the leftover $(-1, 1, -1, 1)$ looks like alternating noise and a second stump can do little; with $\eta = 0.5$ the leftover $(-2, 0, 0, 2)$ still carries the slope inside each leaf, so T_2 can split at 1.5 or 3.5 and recover the trend: later trees re-partition the space instead of patching the first tree's regions. (ii) $F_0 + \eta \sum_m T_m$ is many small corrections to slightly different residuals: sample noise is fitted a fraction η at a time and averaged out across rounds. (iii) Capacity grows in small steps, so the validation curve is smooth and early stopping is reliable. Price: $\eta \propto 1/M$, more rounds. Smaller η usually generalises slightly better, but a few configurations do not prove "always": M and depth must be re-tuned with it, and the gain may not be worth the compute.

Q3 (variant, new numbers). $x = 1, 2, 3, 4$; $y = 2, 4, 6, 12$; stumps; $\eta = 0.5$. First stump splits at $x \le 2$. Then compare the three possible second splits by the SSE of the residual fit, give F_2 and the training MSE of F_0, F_1, F_2.
Answer. $F_0 = 24/4 = 6$; $r_1 = (-4, -2, 0, 6)$. Leaves $\{1, 2\}$: -3; $\{3, 4\}$: $+3$. $F_1 = 6 + 0.5\,T_1 = (4.5, 4.5, 7.5, 7.5)$; $r_2 = (-2.5, -0.5, -1.5, 4.5)$. Second stump: $x \le 1$: means $-2.5 \mid 0.833$, SSE $0 + 20.7 = 20.7$; $x \le 2$: $-1.5 \mid 1.5$, SSE $2 + 18 = 20$; $x \le 3$: $-1.5 \mid 4.5$, SSE $2 + 0 = 2$, best. $F_2 = F_1 + 0.5\,T_2 = (3.75, 3.75, 6.75, 9.75)$. MSE: F_0: $(16 + 4 + 0 + 36)/4 = 14$; F_1: $(6.25 + 0.25 + 2.25 + 20.25)/4 = 7.25$; F_2: $(3.06 + 0.06 + 0.56 + 5.06)/4 = 2.19$. The second stump chose a different split from the first: that is the global adaptability a single deep tree lacks.

6 Boosting for Binary Classification and the XGBoost Objective

Boost the logit

Probabilities cannot absorb tree outputs ($0.9 + 0.4 = 1.3$). The ensemble therefore produces an unconstrained **logit** $F^{(i)} \in (-\infty, \infty)$, maps it through the sigmoid $\hat{p}^{(i)} = \sigma(F^{(i)}) = 1/(1+e^{-F^{(i)}})$, and uses the binary cross-entropy written in F,

$$\ell(y, F) = -\big[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})\big], \qquad \hat{p} = \sigma(F).$$

Trees are added to F: $F_m = F_{m-1} + \eta T_m$, and $\hat{p}$ is recomputed. The negative gradient $y - \hat{p}$ is the classification analogue of the residual.

Gradient and Hessian in the logit: $g = \hat{p} - y$, $h = \hat{p}(1 - \hat{p})$

Step 1, sigmoid derivative. $\sigma = (1 + e^{-F})^{-1}$, so $\sigma' = e^{-F}(1 + e^{-F})^{-2} = \sigma \cdot \frac{e^{-F}}{1+e^{-F}} = \sigma(1 - \sigma)$, because $\frac{e^{-F}}{1+e^{-F}} = 1 - \frac{1}{1+e^{-F}}$. *Step 2, loss in $\hat{p}$.* $\partial\ell/\partial\hat{p} = -y/\hat{p} + (1 - y)/(1 - \hat{p})$. *Step 3, chain rule.*

$$g = \frac{\partial \ell}{\partial F} = \Big(-\frac{y}{\hat{p}} + \frac{1 - y}{1 - \hat{p}}\Big)\hat{p}(1 - \hat{p}) = -y(1 - \hat{p}) + (1 - y)\hat{p} = -y + y\hat{p} + \hat{p} - y\hat{p} = \hat{p} - y.$$

Step 4. g depends on F only through $\hat{p}$ and y is constant, so $h = \partial g/\partial F = d\hat{p}/dF = \hat{p}(1 - \hat{p})$.
Reading: h is the Bernoulli variance, at most 0.25 (at $\hat{p} = 0.5$, the model unsure, the loss curves most) and $\to 0$ when the model is confident (loss flat in F). For $y = 1$ and $\hat{p} \to 0$ (confidently wrong), $-g \to 1$ is largest while $h \to 0$: a big residual with almost no curvature. Quiz check: $\hat{p} = 0.9$, $y = 0$ gives $g = 0.9$, $h = 0.09$.

Newton leaf value: residual divided by curvature

The structure of T_m is grown by squared-error splits on the pseudo-residuals $y - \hat{p}$. With the leaves fixed, the exact output $w_q = \arg\min_w \sum_{i \in S_q} \ell(y^{(i)}, F^{(i)} + w)$ is a sum of log-sigmoids with no closed form (a numerical **line search** per leaf, the bottleneck). Expand each loss to second order around the current logit, $\ell(F + w) \approx \ell(F) + g\,w + \frac{1}{2}h\,w^2$, and sum over the leaf, adding an L_2 penalty $\frac{\lambda}{2}w^2$:

$$\phi(w) = G\,w + \tfrac{1}{2}(H + \lambda)\,w^2, \qquad G = \sum_{i \in S_q} g^{(i)}, \quad H = \sum_{i \in S_q} h^{(i)}.$$

A convex parabola ($h > 0$); $\phi'(w) = G + (H + \lambda)w = 0$ gives

$$w^* = -\frac{G}{H + \lambda} = \frac{\sum_{i \in S_q}(y^{(i)} - \hat{p}^{(i)})}{\sum_{i \in S_q} \hat{p}^{(i)}(1 - \hat{p}^{(i)}) + \lambda}, \qquad \phi(w^*) = -\frac{1}{2}\frac{G^2}{H + \lambda}.$$

$\lambda = 0$ is the plain Newton step $\sum(-g)/\sum h$. Numerator: total residual, how far and which way the probabilities are off. Denominator: total curvature. A leaf of confidently-wrong observations has $H \approx 0$ and the step *explodes*; λ is a floor that caps it ($|w^*| \le |G|/\lambda$), most active exactly where H is smallest. A first-order expansion ($\ell + gw$) is linear in w, has no minimum, and would still need the line search: the $\frac{1}{2}hw^2$ term is what buys the closed form.

Tutorial 2, Q5(b): the leaf by hand

Leaf $(y, \hat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$. $g = \hat{p} - y$: $-0.8, -0.6, +0.8$, $G = -0.6$. $h = \hat{p}(1-\hat{p})$: $0.16, 0.24, 0.16$, $H = 0.56$. Unregularised $w^* = 0.6/0.56 = 1.071$; $\lambda = 1$: $0.6/1.56 = 0.385$. With $\eta = 0.1$ every logit in the leaf moves by $0.1 \times 0.385 = 0.0385$ (0.107 unregularised). Sign check: two under-predicted positives, one over-predicted negative, net upward. The leaf value is the h-weighted mean of the targets $-g/h = 5, 2.5, -5$ ($\sum h(-g/h)/\sum h = \sum(-g)/\sum h$), not their plain mean 0.83.

XGBoost: one regularised objective for structure and leaf values

XGBoost scores a whole proposed tree with leaves $S_1, \ldots, S_Q$ and outputs w_q by

$$\mathcal{L} = \sum_{q=1}^{Q} \sum_{i \in S_q} \left[g^{(i)} w_q + \tfrac{1}{2} h^{(i)} w_q^2 \right] + \gamma Q + \frac{\lambda}{2} \sum_{q=1}^{Q} w_q^2 :$$

the second-order loss change, plus γ **per leaf**, plus $\frac{\lambda}{2}$ **per squared output**. One number chooses splits *and* outputs, so gradients, Hessians and penalties act while growing the tree (GBM applies them after the structure is frozen). Completing the square: each observation asks for target $-g/h$ with weight h; their product $y - \hat{p}$ is bounded by 1, so a confidently-wrong point cannot dictate a leaf.

Optimal leaf value and the gain of a split

Leaf q contributes $\phi_q(w) = G_q w + \frac{1}{2}(H_q + \lambda)w^2$, minimised at $w_q^* = -G_q/(H_q + \lambda)$ with value $\phi_q(w_q^*) = -\frac{1}{2} G_q^2/(H_q + \lambda)$: "how much loss this leaf removes with its best output", more negative is better. Splitting a parent (G, H) into children $(G_L, H_L), (G_R, H_R)$ with $G = G_L + G_R$, $H = H_L + H_R$ replaces one leaf by two, so the leaf count rises by one and the penalty by exactly γ:

$$\text{gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma, \qquad \text{accept iff gain} > 0.$$

λ sits in every denominator and *shrinks* all outputs, most where H is small; γ is a flat price per extra leaf that *discourages* marginal splits. Further controls: minimum child weight (reject a child with H too small, the direct cure for the exploding step), row and feature subsampling, depth limits.

Tutorial drill

Q1 (Tutorial 2, Q5(a)). Prove $g = \hat{p} - y$ and $h = \hat{p}(1 - \hat{p})$ from $\ell = -y \ln \hat{p} - (1 - y) \ln(1 - \hat{p})$.
Answer. The four steps of the mechanism box: $\sigma' = \sigma(1 - \sigma)$; $\partial \ell / \partial \hat{p} = -y/\hat{p} + (1 - y)/(1 - \hat{p})$; chain rule, multiply by $\hat{p}(1 - \hat{p})$, simplify to $\hat{p} - y$; differentiate again, $h = d\hat{p}/dF = \hat{p}(1 - \hat{p})$. Write every line; the marks are in the cancellation $-y + y\hat{p} + \hat{p} - y\hat{p}$. Consequence: $-g = y - \hat{p}$ is the pseudo-residual, $h > 0$ so the Newton parabola is convex.

Q2 (Tutorial 2, Q5(c)). One observation has $\hat{p}$ very close to 0. Its target $-g/h$ is huge; what happens to h, and why read both together?
Answer. Take $y = 1$: $-g = 1 - \hat{p} \to 1$, $h = \hat{p}(1 - \hat{p}) \to 0$, target $-g/h = 1/\hat{p} \to \infty$: the quadratic model says the loss is flat here, so a residual of 1 needs an enormous step. But the observation's weight in the leaf objective is the same $h \to 0$, so its request is almost ignored; what enters $w^* = \sum(-g)/(\sum h + \lambda)$ is the product $h \cdot (-g/h) = 1 - \hat{p} \le 1$: at most 1 to the numerator, nothing to the denominator. Target alone

says "thousands", weight alone says "irrelevant"; the truth is a pull in the right direction with bounded force. (For $y = 0$, $\hat{p} \to 0$ is confidently *right* and nothing explodes.) A leaf made *only* of such points has $\sum h \approx 0$: that is what λ and minimum child weight are for.

Q3 (Tutorial 2, Q6(a)–(b)). Parent $(G, H) = (-4, 3)$, left $(-3, 1)$, right $(-1, 2)$, $\lambda = 1$, $\gamma = 0.2$. Minimiser of ϕ, minimum values, objective decrease, accept? Largest γ accepted.
Answer. $\phi'(w) = G + (H + \lambda)w = 0 \Rightarrow w^* = -G/(H + \lambda)$, $\phi(w^*) = -\frac{1}{2}G^2/(H + \lambda)$. Parent: $w^* = 1$, $\phi = -\frac{1}{2} \cdot 16/4 = -2$. Left: $w^* = 1.5$, $\phi = -2.25$. Right: $w^* = 1/3$, $\phi = -0.167$. Children -2.417 vs parent -2: improvement 0.417; gain $= 0.417 - 0.2 = 0.217 > 0$, accept; accepted while $\gamma < 5/12 \approx 0.417$. A binary split adds exactly one leaf, so γQ rises by exactly γ (a threshold); λ sits in every denominator and shrinks all outputs (continuous).

Q4 (variant, new numbers). Leaf $(1, 0.3), (1, 0.6), (0, 0.2)$. g, h, G, H; w^* for $\lambda = 0$ and 1; loss removed at $\lambda = 1$; then split first two left, third right, with $\lambda = 1$ and $\gamma = 0.1$, then $\gamma = 0.2$.
Answer. $g = -0.7, -0.4, +0.2$, $G = -0.9$; $h = 0.21, 0.24, 0.16$, $H = 0.61$. $w^* = 0.9/0.61 = 1.475$ ($\lambda = 0$), $0.9/1.61 = 0.559$ ($\lambda = 1$); loss removed $\frac{1}{2}(0.81)/1.61 = 0.252$. Split: $G_L = -1.1$, $H_L = 0.45$; $G_R = 0.2$, $H_R = 0.16$. Gain $= \frac{1}{2}[1.21/1.45 + 0.04/1.16 - 0.81/1.61] - \gamma = \frac{1}{2}[0.834 + 0.034 - 0.503] - \gamma = 0.183 - \gamma$. $\gamma = 0.1$: $0.083 > 0$, split; $\gamma = 0.2$: $-0.017 < 0$, keep one leaf. γ is the price of a leaf.

7 Bagging versus Boosting; Stacking; Mixture of Experts

The two families side by side

Training. Bagging and forests: parallel, independent randomised fits, *averaged*. Boosting: sequential corrections, *summed*; each round needs the last.
Mechanism. Bagging averages unstable deep trees: **variance**. Boosting reduces the current loss with shallow trees: mostly **bias**.
Diversity and controls. Bagging: bootstrap rows and m_{try} subsets; leaf size, M up to the OOB plateau. Boosting: changing residuals; η, depth, M by early stopping, λ, γ.
Noise. Bagging is robust and more trees never overfit more; boosting treats every residual as signal and can chase noise and wrong labels.
Tendencies, not laws. Diagnose first: an unstable strong learner wants bagging or a forest; a structured residual pattern wants boosting; different model families with complementary errors want validated voting.

When no ensemble helps

All models miss the same information ($\rho \approx 1$); the models are near copies; boosting is fitting noise or wrong labels; validation data were reused until the choices overfit them; deployment data differ from training data (a forest leaning on destination port learns nothing about new ports from more trees). Ensembles also cost compute, memory and complexity.

Stacking replaces the fixed vote by a learned one: base models (logistic regression, a forest, a boosted model) produce predictions $z_1, \ldots, z_M$ and a simple **meta-model**, typically a regularised logistic regression $\hat{p}_{\text{stack}} = \sigma(\beta_0 + \sum_m \beta_m z_m)$, learns one global set of weights (soft voting is $\beta_m = 1/M$). It must be trained on **out-of-fold** predictions: K folds, base models fitted on $K - 1$ and predicting the held-out fold, so every training row gets a z from models that never saw it. In-sample predictions leak: a deep forest predicts its own rows almost perfectly, so the meta-model learns to trust whichever base model overfits most. For a new input, refit the base models on all the data and pass their outputs through the meta-model.

Mixture of experts makes the weights depend on the input: $p(y \mid x) = \sum_m g_m(x)\, p_m(y \mid x)$ with a **gate** $g_m(x) = \text{softmax}_m(a_m(x))$, $a_m(x) = \beta_m^\top x + b_m$, so each expert can own a region of feature space. Gate and experts are trained *jointly* on the mixture likelihood; experts specialise where the gate sends them. Top-k routing evaluates only the k best-scored experts (conditional computation, the device behind very large language models). Risk: **expert collapse**, one expert attracts everything; load-balancing penalties counter it. Stacking: fixed weights learned after the

base models; mixture: input-dependent weights, learned with the experts.

Tutorial drill

Q1 (Tutorial 2, Q8(b)). OOB predictions versus out-of-fold predictions; how to produce the out-of-fold predictions for a stacking meta-model.
Answer. OOB is a by-product of bagging: i is predicted by the $\approx 0.368M$ trees whose bootstrap omitted it, inside one ensemble, no extra fitting. Out-of-fold uses an explicit K-fold split and works for any base model: for each fold, fit every base model on the other $K-1$ folds and predict the held-out fold; concatenate the K blocks so every training row has one prediction per base model from a model that never saw it; fit the meta-model on those (z, y). The meta-model then judges each base model on unseen data, so memorising is not rewarded.

Q2 (Tutorial 2, Q9(a), condensed). A deep tree changes a lot when the sample changes; a forest is stable but leans on destination port; deployment will see new ports. What can bagging, a forest and boosting each help with, and how to separate "gain from the ensemble" from "gain from the port"?
Answer. Instability across samples is variance: bagging averages it away, a forest also decorrelates the trees (m_{try}); boosting helps with a structured residual (bias), not with instability. None repairs reliance on a feature whose values will change: shared missing information, $\rho \approx 1$ across trees. Experiment: a 2×2 design, each model with and without the port column, same rows for all; the ensemble effect is the difference within a column, the port effect within a row. Validate on services held out of training (group split by service), not a random split, to see the deployment case.

Q3 (variant). Give one case where adding models to an ensemble cannot reduce the error, with the formula that says so.
Answer. Every model misses the same feature, so at every x all errors share a direction: $\rho \to 1$ and $\text{Var}(\bar{e}) = \sigma^2(\rho + (1-\rho)/M) \to \sigma^2$ for any M. Same for M copies of one model or a label error present in every bootstrap sample. The fix is new information or a new model family, not more of the same.

The mistakes that cost points

- Writing σ^2/M for the variance of the average: that is the $\rho = 0$ case; the answer is $\sigma^2(\rho+(1-\rho)/M)$ with the floor $\rho\sigma^2$ named.
- Forgetting the weights: N_t/N in the MDI credit, N_L/N_t and N_R/N_t in the impurity decrease, h in the Newton leaf (the h-weighted mean of $-g/h$, never the plain mean).
- Sign confusion: $g = \hat{p} - y$ is the gradient, $r = y - \hat{p}$ the pseudo-residual; $w^* = -G/(H+\lambda)$, so under-predicted positives ($G < 0$) get a positive output. Apply η in the update, after the fit.
- "OOB error is a test error"; "importance shows what causes attacks". OOB is internal validation that selection overfits; importance is reliance of one fitted model, masked by correlation, silent on causation.

Chapter 5 in five lines

1. Combining helps when models are competent and their errors differ: $\text{Var}(\bar{e}) = \sigma^2(\rho + (1-\rho)/M)$, floor $\rho\sigma^2$. Hard voting counts heads (robust); soft voting averages posteriors (margins, needs calibration).
2. Trees: boxes with one constant each (mean, median, majority); greedy splits by impurity decrease $I(S) - \frac{|S_L|}{|S|}I(S_L) - \frac{|S_R|}{|S|}I(S_R)$, Gini $1 - \sum \hat{p}_c^2$; flexible, cheap, high variance.
3. Bagging: N draws with replacement, $(1-1/N)^N \to 0.368$ out of bag; OOB error is free validation, not a test set. Forest: m_{try} candidates per node, feature absent from a root with probability $1 - m_{\text{try}}/d$, from all M roots $(1 - m_{\text{try}}/d)^M$; weaker trees, smaller ρ, stronger forest; tune at forest level.
4. Importance: MDI credit $\frac{N_t}{N}[\,\cdot\,]$ summed, averaged, normalised, biased to many-valued features; permutation importance = error after shuffling minus before, no retraining; masking, not causation.
5. Boosting: $F_m = F_{m-1} + \eta T_m$, $F_0 = \bar{y}$, trees fit the negative gradient (residuals), leaf = mean residual, $\eta < 1$ leaves structure for later trees. Logit boosting: $g = \hat{p} - y$, $h = \hat{p}(1-\hat{p})$, Newton leaf $-G/(H+\lambda)$, gain $\frac{1}{2}[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}] - \gamma$. Stacking: fixed weights from out-of-fold predictions; mixture of experts: input-dependent weights, learned jointly.